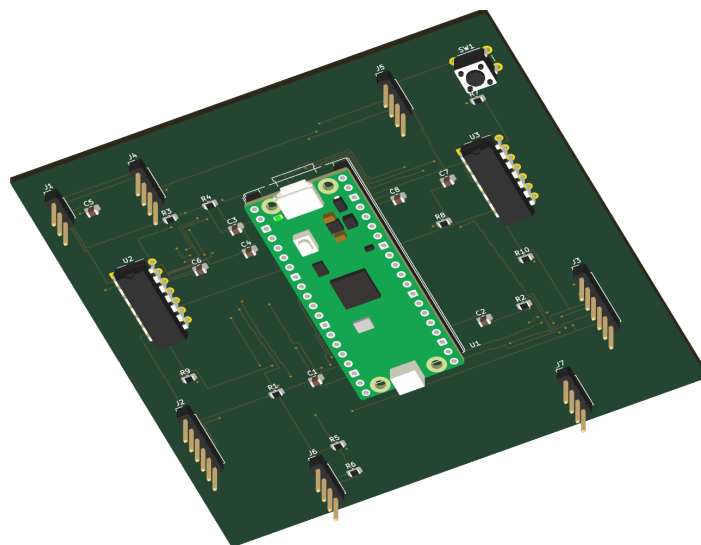


Traction-aware warehouse rover

Measured virtual-prototype demo and phase report | 2 October 2026

A software-only project combining linear PI/LQR, nonlinear tracking, traction supervision, real ROS 2 messaging, and an editable virtual controller PCB. The interactive website, narrated demo, editable KiCad project and phase evidence are delivered through GitHub. Automated GitHub verification tests remain manual at the user's request.

Executed evidence	Result
Matched simulations	540 unique cases; 540 completed routes; all states finite
Local regressions	15 passed; zero failures/errors in saved JUnit results
Recorded wall-clock tests	Four approximately 16 s runs; zero 20 ms compute-budget misses
ROS 2 integration	DDS/TF, controller-selection service, bag record/replay executed
KiCad / circuit	KiCad 10.0.6; ERC 0; DRC 0 errors / 0 warnings; RC tau 1.00116 ms
Gazebo contact	Torque commands moved the body; low-friction wheel/body divergence increased



Actual local KiCad render of the 29-component Pico carrier. Library keepouts, pad clearance and via drills are respected. Local rule checks pass; manufacturing readiness and physical performance require further validation.

Results by phase

Phase	Status	Work and verified result
1	Verified	Requirements and experiment matrix Fixed sensor/control boundaries, rates, payloads and friction cases.
2	Verified	Robot model and warehouse Mathematical dynamics checks and real Gazebo dry/low-friction contact bridge passed.
3	Verified	ROS 2 basics and integration Actual DDS traffic, TF, controller-selection service, bag record/replay.
4	Verified	Linear PI / LQR Controllable and stable design point. Worst dry LQR RMSE: 8.76 mm.
5	Verified	Nonlinear control / traction All runs finite and complete. Low-traction slip reduction: 18.48%; 20% target not met.
6	Verified	Virtual PCB and circuit KiCad 10 ERC 0, DRC 0, connectivity 0; source, renders, Gerbers and RC simulation.
7	Verified	Software and paced tests 15 local tests; 540 completed routes; 4 recorded live runs; 5 fault cases.
8	Verified	GitHub demo and profile Repository, interactive Pages website, phase report and profile project link delivered.

A phase marked Verified has executed evidence for its stated software scope. The 20% slip-reduction hypothesis was not met. Zero local PCB violations does not establish physical performance or formal standards conformance.

Project purpose and standards intent

Slippery patches and varying payloads can make wheel speed a poor proxy for body motion. The project addresses that issue with independent velocity estimation and torque/acceleration derating, using an inspectable digital model and matched comparisons.

Design intent is mapped to IPC-2221/IPC-2152 (PCB design), ISO 12100 (hazards), ISO 3691-4 (driverless trucks), ISO 13849-1 (fault-response separation), and ROS REP 103/105 (units/frames). Paid standard texts were not reviewed clause by clause. This is scope-level alignment, not conformance or certification.

Model and control methods

Item	Implemented setting
Plant	Independent x/y/yaw, body speed/yaw rate, wheel speeds and motor currents
Mechanical / electrical	12 kg base; 0/5/10 kg payload; 75 mm wheels; 380 mm axle; 24 V; R 1.5 ohm; L 3 mH; kt/ke 0.12
Sampling rates	Plant 1 kHz; wheel PI 200 Hz; estimator / outer control 50 Hz
Traction scenarios	Dry mu 0.8; patch mu 0.1; asymmetric left/right friction
Sensor boundary	Noisy delayed independent pose, quantized encoders and IMU estimates; truth only for sensor generation/evaluation
Board model	Signed centered 12-bit ADC; 12-bit PWM; 10 ms command delay; 1 ms RC; 100 ms watchdog; independent latched 16 A trip

Linear: wheel PI with anti-windup, plus moving-reference LQR at 0.45 m/s. $Q = \text{diag}(3.24, 6.25, 2.25)$, $R = I$. Controllability and negative closed-loop eigenvalue real parts are checked. Stopping uses a separate bounded alignment mode.

Nonlinear: $v = v_r \cos(\epsilon) + 1.8 e_x$; yaw rate = $w_r + 5 v_r e_y + 2 \sin(\epsilon)$. A nominal no-slip Lyapunov argument supports local behavior; it does not prove global convergence with saturation, delay, slip or faults.

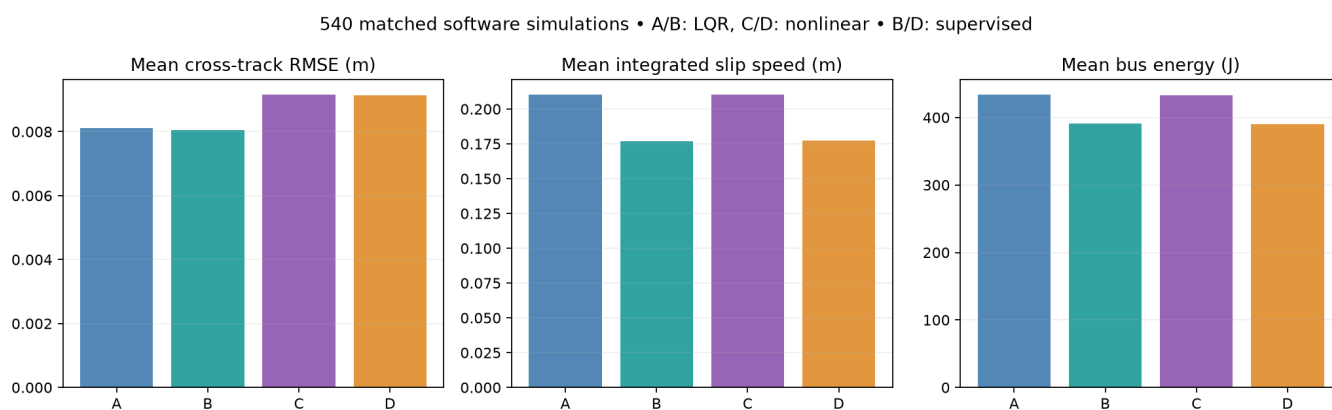
Traction supervision: sensor-derived slip threshold 0.22 with 60 ms persistence; recovery threshold 0.12. Active derating lowers torque from 1.44 to 0.32 Nm and acceleration from 1.8 to 0.25 m/s². The route uses a shared 2.2-4.6 s acceleration/braking demand to exercise traction.

Model limits: no lateral tire-slip, tipping, suspension, thermal or detailed battery model. Coasting follows loss of drive; no emergency or parking brake is modeled.

Matched controller comparison

540 cases = 4 variants x 3 surfaces x 3 payloads x 3 routes x 5 noise seeds. A/B use LQR, C/D nonlinear tracking. B/D enable supervision. All four groups completed 135/135 routes.

Variant	Mean RMSE (mm)	Mean integrated slip (m)	Mean energy (J)
A	8.113	0.21029	434.15
B	8.060	0.17676	391.50
C	9.165	0.21050	433.12
D	9.130	0.17741	390.55



Low-traction integrated-slip reduction, calculated only on patch/asymmetric cases: **18.76% LQR (A to B)** and **18.48% nonlinear (C to D)**. The bars/table above average all surfaces, including dry cases; their percent difference therefore differs.

The predeclared 20% reduction hypothesis was not met. All recorded runs remained finite and completed, and supervised dry runs had no supervisor activation. This is evidence for the modeled scenarios, not a general guarantee across floors or robots.

ROS integration and warehouse validation

Actual ROS check	Saved result
Observed messages, including replay	Odometry 881; commands 881; TF 881
Bag replay	430 replayed odometry messages; metadata created
Controller-selection service	SetBool call successful
Body displacement	3.523582 m
Test elapsed, including replay	11.861 s

The graph contains separate PlantNode and ControllerNode processes/classes, timestamped odometry, /tf, /clock, wheel-state/IMU messages, diagnostics, parameters, a controller-selection service and a reset service. The test used actual ROS 2 Jazzy middleware and recorded/replayed a bag. The explanatory graph in the video is an illustration of that tested data path, not a live ROS screen capture.

Gazebo: the corrected inertia ran successfully in saved run 36900851054. Both worlds received 0.8 Nm wheel effort through ros_gz_bridge. Dry displacement: 1.335 m; low-friction displacement: 0.705 m. Wheel/body divergence increased from 0.327 to 8.319 m/s. Evidence was recovered from the existing successful run; no new GitHub test was launched. This checks contact response separately from the 540-case mathematical-plant study.

ROS reproduction commands

On a ROS 2 Jazzy Linux environment, source /opt/ros/jazzy/setup.bash; install the root Python package; run colcon build --symlink-install from ros_ws; source install/setup.bash; then run python tools/ros_smoke.py from the project root. See the repository workflow for exact environment setup. A fresh bag output folder is required for repeated recordings.

Key interfaces: /amr/observation, /reference, /drive/wheel_targets, /odom, /tf, /clock, /controller/use_nonlinear and /amr/reset. SI units and map -> odom -> base_link frame ownership follow the documented REP design intent.

Virtual PCB, analog circuit and local KiCad

The editable carrier uses a Raspberry Pi Pico, 74HC08 gating and a 74HC74 latch, seven connectors, RC current-sense filtering, bus-voltage divider, I2C pullups and decoupling. Motor current flows in external power drivers. Their centered 0.1 V/A current outputs and active-low driver faults are interface assumptions.

Local artifact/check	Actual result
KiCad version	10.0.6
Carrier geometry	96 x 96 mm; two copper layers; 29 components; 29 connected nets + 18 no-connect nets
Schematic ERC	0 violations
PCB connectivity	0 unconnected items
PCB DRC	0 errors; 0 warnings
Exports	Top and angled component renders; board-only STEP; schematic SVG
Analog RC expected / measured	1.00000 ms / 1.00116 ms
Analog final voltage	2.64909 V for a 1.65 -> 2.65 V sense-source step

DRC finding	Count
No layout violations	0

The local router respects library copper keepouts, rotated pad bounds, separate copper clearance, and drill spacing. Through-hole pads can connect on either copper layer; existing vias are reused and collinear segments are merged. DRC and ERC pass without suppressing the previous keepout, shorting or drilling defects. The full project includes editable source, Gerbers, drills and a STEP export. Physical fabrication and assembled-board validation remain outside this virtual project.

Ngspice simulates the RC measurement chain driven by an ideal external-driver sense source. Digital trip/watchdog/ADC/PWM behavior is checked in the virtual board model. These are separate evidence sources; a complete assembled-board SPICE simulation, EMI validation and physical timing measurements were not performed.

Live pacing and injected faults

Variant	Wall duration (s)	p99 / max compute (ms)	Misses / wake p99 (ms)
A	15.982	1.923 / 2.782	0 / 0.558
B	15.982	2.132 / 13.116	0 / 0.556
C	15.981	1.967 / 2.476	0 / 0.557
D	15.982	2.133 / 2.605	0 / 0.574

The 20 ms compute budget was met in all four saved Windows runs. Wall duration is approximately 15.98 s because the first simulated block starts at wall time zero and the last block begins at 15.98 s. Wake lateness is separate from compute duration. This does not demonstrate a hard real-time kernel or guaranteed future deadlines.

Fault, injected at 5 s	Drive off at (s)	Response (ms)	Modeled coast (m)
overcurrent	5.000	0	1.104
command_timeout	5.096	96	1.430
encoder_dropout	5.060	60	1.296
localization_stale	5.140	140	1.442
imu_bias	5.000	0	1.104

All five fault cases remained finite and disabled drive. These are separate dry, straight-route cases. The fault animation in the video replays a recorded Linux curved-route overcurrent test; its trace is not the source of the dry-case coast distances in this table.

Power cut permits electrical current decay and body coasting. The distance starts at fault injection and accumulates subsequent simulated travel. These values are not safe-stop or certified industrial braking distances. No physical brake is modeled.

Delivered files and reproduction

Deliverable	Project-relative location
Narrated H.264 / AAC demo	output/video/traction-aware-amr-demo.mp4
Chapter transcript / subtitles	output/video/transcript.md; traction-aware-amr-demo.srt
This PDF / evidence hashes	output/pdf/traction-aware-amr-report.pdf; evidence.json
Editable schematic / PCB	hardware/kicad/traction_carrier.kicad_sch; traction_carrier.kicad_pcb
KiCad renders / geometry	hardware/exports/carrier-top.png; carrier-isometric.png; carrier.step
Saved measurements	results/matrix.json; live_A/B/C/D.json; faults.json; ros_smoke.json; spice.json; erc.json; drc.json

Local software commands: install requirements.txt in Python 3.12 with a compatible native compiler; `python -m pytest -q`; `python -m amr.experiments matrix --workers 4`; `python -m amr.experiments live --variant D --seconds 16`; `python -m amr.experiments faults`. These commands are for reproduction; they were not rerun for this video assembly.

Video reproduction uses `tools/demo_chapters.py`, `tools/narrate.py` (installed Windows SAPI voice through `comtypes`), and `tools/build_demo.py --video`. PDF reproduction uses `tools/build_demo.py --report`. Additional local authoring dependencies: Pillow, imageio-ffmpeg, comtypes, reportlab and pypdf. Video animations use recorded traces and actual local KiCad renders. Narration is computer-generated; caption callouts and the transcript are included.

Project repository: github.com/Snow-Warrior07/traction-aware-amr. Interactive demo: snow-warrior07.github.io/traction-aware-amr/. GitHub profile links to the project. The website includes downloads for this report, KiCad project, Gerbers, evidence and narrated demo. `output/publication.json` records the verified deployment.

Evidence and standards references

The accompanying `evidence.json` records SHA-256 hashes of the exact input measurements and KiCad files used. Principal evidence: 540-case matrix, 15-test JUnit output, four local paced-run JSON/CSV pairs, five-fault JSON, ROS smoke JSON, ngspice JSON/data, and local KiCad ERC/DRC JSON. A green job status or an unexecuted source file was not used to infer an engineering pass.

Primary scope references: IPC design standards (ipc.org/ipc-design-standards/); ISO 12100 (iso.org/standard/51528.html); ISO 3691-4 (iso.org/standard/83545.html); ISO 13849-1 (iso.org/standard/73481.html); ROS REP 103/105 (github.com/ros-infrastructure/rep); KiCad CLI reference (docs.kicad.org/10.0/en/cli/cli.html). Existing `docs/standards.md` gives the project hazard register and limits.